

Smallest Subsequence Explanation

Absolutely. Let's restart this from **zero**, because the confusing part is that there are **three separate ideas** working together:

1. **last occurrence / frequency** → tells us whether we can safely remove a character.
2. **used[]** → tells us whether we already have that character in our answer.
3. **stack/string** → maintains the current answer and allows us to remove characters from the end.

I'll first show the **brute force**, then explain the greedy idea, and finally explain all **4 versions**.

1. First understand the problem

Example:

```
s = "cbacdcbc"
```

We need:

- every distinct character exactly once
- preserve the relative order from **s**
- among all valid subsequences, choose lexicographically smallest

Answer:

```
"acdb"
```

Why not "abcd"?

Because "abcd" cannot be formed as a subsequence of:

```
c b a c d c b c
  ↑ ↑ ↑ ↑
  a b c d
```

The **b** that could come after **a** is at index 6, but **c** after that is at index 7, while **d** is at index 4. So "acdb" is the valid smallest arrangement.

2. Brute Force Approach

Before understanding the greedy solution, let's understand what the brute force is actually doing.

For a string of length **n**, there are:

```
2^n
```

subsequences.

For each subsequence:

1. Check whether every character is unique.
2. Check whether its length equals the number of distinct characters in **s**.
3. If valid, compare it with our best answer.

For example:

```
s = "abc"
```

Subsequences:

```
""
"a"
"b"
"c"
"ab"
"ac"
"bc"
"abc"
```

Only "abc" contains all distinct characters exactly once.

Brute force code

```
C++
```

```
class Solution {
public:
    string ans;

    void generate(string &s, int index, string &cur, vector<bool> &used) {
        if (index == s.size()) {
            // We need one occurrence of every distinct character
            if (cur.size() == used.size()) {
                // not actually useful with fixed 26 array
            }

            return;
        }

        // Take s[index]
        generate(s, index + 1, cur, used);

        // Don't take s[index]

    }

    string smallestSubsequence(string s) {
        // Number of distinct characters
        set<char> distinct(s.begin(), s.end());

        ans = "";

        function<void(int, string&)> dfs =
            [&](int index, string &cur) {
                if (index == s.size()) {
                    if (cur.size() != distinct.size())
                        return;

                    vector<bool> seen(26, false);

                    for (char c : cur) {
                        if (seen[c - 'a'])
                            return;

                        seen[c - 'a'] = true;
                    }

                    if (ans.empty() || cur < ans)
                        ans = cur;

                    return;
                }

                // Don't take current character
                dfs(index + 1, cur);

                // Take current character
                cur.push_back(s[index]);

                dfs(index + 1, cur);

                cur.pop_back();
            };

        string cur;
        dfs(0, cur);

        return ans;
    }
};
```

However, there's an important problem with this brute force:

2^n

subsequences.

So if $n = 100$, this becomes completely impractical.

That's why we need the greedy + stack solution.

3. The Core Greedy Idea

Suppose our current answer is:

"cb"

and current character is:

"a"

We have:

$c > a$

So we'd rather have:

```
"ab"
```

than:

```
"cba"
```

But can we remove **c**?

Only if we know that another **c** appears later.

For example:

```
s = "cbaac"
```

Current:

```
stack = "cb"
current = 'a'
```

We want to remove **b** because:

```
b > a
```

But before removing **b**, we need to know:

"Can I get another **b** later?"

If yes → remove it.

If no → we MUST keep it.

That's where **last** and **freq** come in.

4. What exactly does `last[]` mean?

Suppose:

```
s = "cbacdcbc"
```

Indices:

```
index: 0 1 2 3 4 5 6 7
        c b a c d c b c
```

Last occurrence:

```
a → 2
b → 6
c → 7
d → 4
```

So:

```
C++
last['b' - 'a'] = 6;
```

means:

The last **b** is at index 6.

Now we're currently at index 3.

If we have **b** in our stack:

```
C++
last['b' - 'a'] > 3
```

becomes:

```
6 > 3
```

True.

Therefore:

There is another **b** after the current position.

So we can remove b.

5. What exactly does `freq[]` mean?

Instead of storing the **position of the last occurrence**, we can store the **number of occurrences remaining**.

For:

```
s = "cbacdcbc"
```

Initial frequency:

```
a = 1
b = 2
c = 4
d = 1
```

When processing a character, we do:

```
C++
freq[c - 'a']--;
```

Why?

Because the current character is no longer in the future.

For example, at index 0:

```
c
```

Initially:

```
freq[c] = 4
```

After processing this c:

```
freq[c] = 3
```

That means:

There are 3 more cs somewhere later.

Therefore:

```
C++
freq[c] > 0
```

means:

I can find this character again later.

6. last vs freq — the important difference

They answer almost the **same question**, but store different information.

last

Stores:

Where is the final occurrence?

```
C++
last[c]
```

Example:

```
last[c] = 7
```

Then we ask:

```
C++
last[c] > i
```

freq

Stores:

How many copies are still remaining?

```
C++
freq[c]
```

Example:

```
freq[c] = 2
```

Then we ask:

```
C++
freq[c] > 0
```

Think about it this way

```
last: "Where is my last chance?"
freq: "Do I still have another chance?"
```

That's the fundamental difference.

7. Now the used[] array

This is a completely different concept.

Suppose:

```
s = "cbacdcbc"
```

Imagine our current answer is:

```
"ac"
```

Then:

```
used[a] = true
used[c] = true
```

and:

```
used[b] = false
used[d] = false
```

So **used** answers:**Have I already put this character into my answer?**It does **NOT** answer whether the character exists later.That's the job of **freq** or **last**.**8. Why do we skip when used[c] == true?**

Suppose:

```
s = "cbacdcbc"
```

We've already added:

```
c
```

to our answer.

Later we encounter another:

```
c
```

But the answer must contain **distinct characters**.

We cannot do:

```
"acc"
```

because c appears twice.

So:

```
C++
if (used[c - 'a'])
    continue;
```

means:

"I already have this character in my answer. I don't need another copy."

9. VERY IMPORTANT: used and freq are NOT the same

This is probably the biggest source of confusion.

Suppose:

```
s = "cbacdcbc"
```

At some point:

```
used[c] = true
freq[c] = 2
```

This means:

```
used[c] = true
```

→ I already have c in my answer.

And:

```
freq[c] = 2
```

→ There are still two more cs in the remaining string.

Completely different information.

10. Why do we sometimes set used[top] = false?

This is another VERY important point.

Suppose:

```
stack = "cb"
current = 'a'
```

We see:

```
b > a
```

and another b exists later.

So:

```
"cb"
```

becomes:

```
"c"
```

We popped b.

Therefore:

```
C++
used['b' - 'a'] = false;
```

Why?

Because **b** is **no longer in our answer**.

So now another **b** is allowed to enter later.

11. Now let's understand Version 1

Last occurrence + stack<char>

```
C++
class Solution {
public:
    string smallestSubsequence(string s) {
        vector<int> last(26, -1);

        // Find last position of every character
        for (int i = 0; i < s.size(); i++) {
            last[s[i] - 'a'] = i;
        }

        vector<bool> used(26, false);
        stack<char> st;

        for (int i = 0; i < s.size(); i++) {
            char c = s[i];

            // Already present
            if (used[c - 'a']) {
                continue;
            }

            // Remove larger characters
            // if they appear again later
            while (!st.empty() &&
                st.top() > c &&
                last[st.top() - 'a'] > i) {

                used[st.top() - 'a'] = false;
                st.pop();
            }

            st.push(c);
            used[c - 'a'] = true;
        }

        string ans;

        while (!st.empty()) {
            ans.push_back(st.top());
            st.pop();
        }

        reverse(ans.begin(), ans.end());

        return ans;
    }
};
```

The important line

```
C++
last[st.top() - 'a'] > i
```

Suppose:

```
stack top = 'b'
current = 'a'
```

and:

```
last[b] = 6
i = 2
```

Then:

```
6 > 2
```

So:

Another **b** exists later.

Therefore we can safely remove **b**.

12. Version 2: last occurrence + string

This is exactly the same algorithm.

The only difference is:

```
C++
stack<char> st;
```

becomes:

```
C++
string st;
```

Because a string can behave like a stack.

Stack version

```
C++
st.push(c);
st.top();
st.pop();
```

String version

```
C++
st.push_back(c);
st.back();
st.pop_back();
```

So:

```
C++
class Solution {
public:
    string smallestSubsequence(string s) {
        vector<int> last(26, -1);

        for (int i = 0; i < s.size(); i++) {
            last[s[i] - 'a'] = i;
        }

        vector<bool> used(26, false);

        string st;

        for (int i = 0; i < s.size(); i++) {
            char c = s[i];

            if (used[c - 'a']) {
                continue;
            }

            while (!st.empty() &&
                   st.back() > c &&
                   last[st.back() - 'a'] > i) {
                used[st.back() - 'a'] = false;
                st.pop_back();
            }

            st.push_back(c);
            used[c - 'a'] = true;
        }

        return st;
    }
};
```

Notice something important

Compare:

```
C++
last[st.top() - 'a'] > i
```

with:

```
C++
last[st.back() - 'a'] > i
```

Only:

```
top()
```

became:

```
back()
```

Everything else is identical.

13. Version 3: frequency + stack

Now we're changing the **first part**.

Instead of:

```
C++
last
```

we use:

```
C++
freq
```

First:

```
C++
vector<int> freq(26, 0);
for (char c : s) {
    freq[c - 'a']++;
}
```

Then while processing:

```
C++
freq[c - 'a']--;
```

This tells us how many copies are still in the future.

Code

```
C++
class Solution {
public:
    string smallestSubsequence(string s) {
        vector<int> freq(26, 0);
        for (char c : s) {
            freq[c - 'a']++;
        }
        vector<bool> used(26, false);
        stack<char> st;
        for (char c : s) {
            // Current character is now processed
            freq[c - 'a']--;
            // Already in answer
            if (used[c - 'a']) {
                continue;
            }
            while (!st.empty() &&
                st.top() > c &&
                freq[st.top() - 'a'] > 0) {
                used[st.top() - 'a'] = false;
                st.pop();
            }
            st.push(c);
            used[c - 'a'] = true;
        }
        string ans;
        while (!st.empty()) {
            ans.push_back(st.top());
            st.pop();
        }
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

14. Why freq-- comes BEFORE the used check

This is subtle.

Suppose:

```
s = "cbacdcbc"
```

We're processing a c.

Before processing:

```
freq[c] = 4
```

After:

```
C++
freq[c]--;
```

we have:

```
freq[c] = 3
```

That's correct because the current c is no longer future.

Now suppose we've already used c:

```
C++
if (used[c - 'a'])
    continue;
```

Even though we don't put this c into the answer, we still need to decrease its frequency because:

This occurrence has been consumed from the input.

That's why:

```
C++
freq--
```

comes first.

15. Version 4: frequency + string

This is my recommended version for learning.

```
C++
class Solution {
public:
    string smallestSubsequence(string s) {
        vector<int> freq(26, 0);
        // Count every character
        for (char c : s) {
            freq[c - 'a']++;
        }
        vector<bool> used(26, false);
        string st;
        for (char c : s) {
            // Current c is no longer in the future
            freq[c - 'a']--;
            // Already have c in answer
            if (used[c - 'a']) {
                continue;
            }
            // Remove characters that:
            // 1. are bigger than c
            // 2. occur again later
            while (!st.empty() &&
                st.back() > c &&
                freq[st.back() - 'a'] > 0) {
                used[st.back() - 'a'] = false;
                st.pop_back();
            }
            st.push_back(c);
            used[c - 'a'] = true;
        }
    }
};
```

```
    }
    return st;
  }
};
```

16. Let's dry run the frequency version

Use:

```
s = "cbacdcbc"
```

Initial:

```
freq:
a = 1
b = 2
c = 4
d = 1
```

i = 0

Current:

```
c
```

Decrease:

```
freq[c] = 3
```

c not used.

Stack empty.

Push:

```
stack = "c"
used[c] = true
```

i = 1

Current:

```
b
```

Decrease:

```
freq[b] = 1
```

b not used.

Check:

```
top = c
c > b    → true
freq[c] > 0 → true
```

Therefore remove c.

```
stack = ""
used[c] = false
```

Push b:

```
stack = "b"
used[b] = true
```

i = 2

Current:

```
a
```

Decrease:

```
freq[a] = 0
```

Check:

```
b > a
freq[b] = 1
```

Both true.

Remove **b**.

```
stack = ""
used[b] = false
```

Push **a**:

```
stack = "a"
used[a] = true
```

i = 3

Current:

```
c
```

Decrease:

```
freq[c] = 2
```

c is not used anymore.

Push:

```
stack = "ac"
used[c] = true
```

i = 4

Current:

```
d
```

Decrease:

```
freq[d] = 0
```

Check:

```
c > d
```

False.

Push:

```
stack = "acd"
```

i = 5

Current:

```
c
```

Decrease:

```
freq[c] = 1
```

But:

```
C++
used[c] == true
```

So:

```
C++
continue;
```

Stack remains:

```
"acd"
```

i = 6

Current:

```
b
```

Decrease:

```
freq[b] = 0
```

b is not currently used.

Check top:

```
top = d
d > b → true

freq[d] > 0
0 > 0 → false
```

Cannot remove d.

Why?

Because there is **no other d later**.

If we remove d, we'd lose d completely.

So keep it.

Next:

```
stack = "acdb"
```

i = 7

Current:

```
c
```

Decrease:

```
freq[c] = 0
```

But:

```
used[c] = true
```

Skip.

Final:

```
"acdb"
```

17. Now the entire algorithm becomes simple

For every character c:

Step 1 — consume current character

```
C++
freq[c]--;
```

Meaning:

This occurrence is no longer in the future.

Step 2 — check duplicate

```
C++
if (used[c])
    continue;
```

Meaning:

I already have this character in my answer.

Step 3 — remove bad characters

```
C++
while (!st.empty() &&
       st.back() > c &&
       freq[st.back()] > 0)
```

This means:

The last character is bigger than the current character, AND I can get that last character again later.

Therefore:

```
C++
pop
```

Step 4 — add current character

```
C++
st.push_back(c);
used[c] = true;
```

18. The three arrays/data structures have three different jobs

This is what I want you to remember:

Variable	Question it answers
freq[c]	Can I get c again later?
used[c]	Do I already have c in my answer?
st	What is my current answer?

And with the last-occurrence version:

Variable	Question
last[c]	Where is the last c?
used[c]	Do I already have c?
st	Current answer

19. The two most important conditions

You'll see these everywhere:

Duplicate check

```
C++
if (used[c])
    continue;
```

Translation:

"I already have this character, so don't add it again."

Removal check

```
C++
st.back() > c &&
freq[st.back()] > 0
```

Translation:

"The previous character is larger than the current one, and I have another copy of it later, so I can remove it."

That's the greedy decision.

20. Why don't we remove a smaller character?

Suppose:

```
stack = "ac"
current = 'b'
```

Should we remove c?

```
c > b
```

No.

Actually:

```
c > b
```

is true, so potentially yes—but only if another c exists later.

But suppose:

```
stack = "ab"
current = 'c'
```

Then:

```
b > c
```

is false.

So we don't remove b.

Because:

```
"abc"
```

is already lexicographically better than something like:

```
"acb"
```

The stack only pops when the new character gives us a lexicographically better ordering.

21. The four versions now

Version 1

```
last occurrence
+
stack<char>
```

Uses:

```
C++
last[st.top() - 'a'] > i
```

Version 2

```
last occurrence
+
string
```

Uses:

```
C++
last[st.back() - 'a'] > i
```

Only stack implementation changed.

Version 3

```
frequency
+
stack<char>
```

Uses:

```
C++
freq[st.top() - 'a'] > 0
```

Only the "does it occur later?" mechanism changed.

Version 4 ★

```
frequency
+
string
```

Uses:

```
C++
freq[st.back() - 'a'] > 0
```

Both implementation choices are the simpler ones.

22. What I recommend you memorize

Don't try to memorize all four separately.

Memorize **one**:

```
C++
class Solution {
public:
    string smallestSubsequence(string s) {
        vector<int> freq(26, 0);
        vector<bool> used(26, false);

        for (char c : s)
            freq[c - 'a']++;

        string st;

        for (char c : s) {
            freq[c - 'a']--;

            if (used[c - 'a'])
                continue;

            while (!st.empty() &&
                   st.back() > c &&
                   freq[st.back() - 'a'] > 0) {
                used[st.back() - 'a'] = false;
                st.pop_back();
            }

            st.push_back(c);
            used[c - 'a'] = true;
        }

        return st;
    }
};
```

And remember this sentence:

If the current character is smaller than the stack top, and the stack top appears again later, remove the stack top. If the current character is already used, skip it.

That's essentially the whole problem.

Complexity

```
Brute force:
Time : O(2^n × n)
Space : O(n)

Greedy:
Time : O(n)
Space : O(1)
```

The greedy is $O(n)$ because although there is a `while` loop, **each character can be pushed and popped at most once**.

ChatGPT can make mistakes. Check important info.